

Câu 1:

a. Hãy định nghĩa nhân bản dữ liệu trong hệ thống phân tán và nêu hai lý do chính mà ta phải triển khai nhân bản.

- Nhân bản dữ liệu trong hệ thống phân tán là quá trình tạo và duy trì nhiều bản sao dữ liệu hệ thống tại nhiều vị trí để đảm bảo tính sẵn sàng, tính tin cậy và khả năng phục hồi của dữ liệu trong hệ thống.
- Hai lý do chính khiến ta phải triển khai nhân bản gồm :
 - Khả năng chịu lỗi: Khi nhân bản, dữ liệu được lưu ở nhiều vị trí, khi một bản sao bị lỗi, máy chủ có thể sử dụng dữ liệu của các bản sao khác, điều này giúp hệ thống chịu lỗi tốt hơn khi một bản sao nào đó xuất hiện lỗi
 - Cải thiện hiệu năng hệ thống : Có nhiều bản sao lưu các vị trí khác nhau nên khách hàng có thể truy cập vào vị trí phù hợp nhất, giúp giảm lượng dữ liệu phải truyền tải xa trên môi trường mạng, các yêu cầu không bị tập trung về máy chủ nên có thể áp dụng cân bằng tải hoặc tính toán phân tán để giảm áp lực lên một máy chủ.

b. Giải thích tại sao việc nhân bản dữ liệu có thể dẫn đến vấn đề nhất quán (consistency). Lấy ví dụ kịch bản cache web để minh họa cách người dùng có thể vẫn nhìn thấy “nội dung cũ” sau khi dữ liệu gốc đã được cập nhật.

- Việc nhân bản dữ liệu có thể dẫn đến vấn đề nhất quán bởi do có nhiều bản sao ở các vị trí khác nhau nên khi một bản sao thay đổi, vấn đề đặt ra là các bản sao khác cũng cần được cập nhật nhưng việc cập nhật cũng cần thời gian và cần xác định được thành phần nào và khi nào cập nhật.
- Ví dụ về kịch bản cache web:
 - Khi một máy khách gửi yêu cầu đến máy chủ, máy chủ cần lấy dữ liệu trong cơ sở dữ liệu và trả ra cho máy khách một html của giao diện. Quá trình này cần mất vài giây để xử lý. Để giảm tải thời gian xử lý cho các lần sau, máy chủ sẽ lưu file html tại một vị trí gọi là cache và các lần truy cập sau của máy khách sẽ được lấy từ cache luôn.
 - Khi máy khách truy cập lần tiếp theo, sự cập nhật chưa được chuyển đến các bản sao khác đã lưu trên cache , nên dữ liệu lúc này có thể là của một bản sao cũ trên cache, để giải quyết vấn đề này thì có thể tạo cơ chế làm tươi cache sau mỗi một khoảng thời gian nhất định.

c. Một dịch vụ e-commerce toàn cầu cần phục vụ đa triệu lượt truy cập mỗi giây. Hãy đề xuất:

1. Cách phân bố bản sao dữ liệu (geo-replication) để tối ưu độ trễ,

2. Cơ chế làm tươi cache (cache invalidation/TTL), sao cho người dùng gần nhất luôn nhận được dữ liệu “đủ mới” mà không quá tải bằng thông.

1. Phân bố bản sao dữ liệu (Geo-replication) để tối ưu độ trễ:

Mục tiêu:

- Tăng tính sẵn sàng, khả năng chịu lỗi
- Giảm độ trễ cho người dùng ở các vùng địa lý khác nhau
- Phân tán tải để xử lý song song

Giải pháp:

- **Triển khai nhân bản dữ liệu tại nhiều vùng địa lý (multi-region replication):** đặt các bản sao dữ liệu (read replica) ở gần với từng cụm người dùng (ví dụ: Mỹ, Châu Âu, Châu Á).
- **Sử dụng cơ chế đọc gần – ghi tập trung:**
 - **Đọc (Read)** từ bản sao gần nhất với người dùng để giảm độ trễ
 - **Ghi (Write)** được chuyển về trung tâm (primary region) để đảm bảo nhất quán
- **Chọn mô hình nhất quán nói lỏng (eventual consistency)** cho phần lớn dữ liệu (ví dụ như thông tin sản phẩm, đánh giá...) để giảm chi phí đồng bộ.
- **Áp dụng thuật toán đồng thuận (như Paxos, Raft hoặc Lamport clock)** nếu cần nhất quán chặt cho các dữ liệu quan trọng (ví dụ: thanh toán, đơn hàng).

Ưu điểm:

- Người dùng luôn truy cập bản sao gần nhất → giảm độ trễ mạng
- Hệ thống có thể mở rộng theo chiều ngang
- Nếu một bản sao bị lỗi, người dùng có thể được chuyển hướng (failover) sang bản sao khác

2. Cơ chế làm tươi cache (cache invalidation/TTL):

Mục tiêu:

- Giảm tải truy vấn trực tiếp vào cơ sở dữ liệu

- Đảm bảo người dùng **nhận dữ liệu đủ mới**, nhưng **không làm tốn băng thông hoặc gây quá tải**

Giải pháp kết hợp:

a. Cache tại nhiều lớp:

- **Edge cache / CDN** (ở gần người dùng): lưu HTML tĩnh, ảnh sản phẩm, thông tin hiếm thay đổi
- **Application-level cache** (Redis, Memcached): lưu dữ liệu sản phẩm, giỏ hàng, kết quả truy vấn

b. TTL (Time-To-Live) thích nghi theo loại dữ liệu:

- TTL ngắn (~10–30s) cho dữ liệu hay thay đổi (giá sản phẩm, kho hàng)
- TTL dài (~5–10 phút) cho dữ liệu ít thay đổi (mô tả sản phẩm, đánh giá)

c. Cache Invalidation có điều kiện:

- Khi có cập nhật giá, kho hàng: gửi thông báo tới các nút cache liên quan để **xóa cache theo key** (cache-by-key invalidation)
- Dữ liệu cập nhật ít (đánh giá, mô tả) chỉ làm mới theo TTL (time-based invalidation)

d. Giảm thiểu băng thông bằng cách không cập nhật toàn bộ:

- Nếu tỷ lệ đọc cao hơn ghi ($N \text{ đọc/s} \gg M \text{ ghi/s}$): cho phép người dùng đọc bản cache chưa mới nhất, chấp nhận trễ nhỏ vài giây
- Dữ liệu rất quan trọng (thanh toán): không dùng cache hoặc buộc cache TTL = 0

d. Giả sử hệ thống bạn quản lý có đặc tính: bản sao được đọc trung bình N lần/s nhưng chỉ ghi M lần/s, trong đó $N \ll M$.

1. Phân tích chi phí mạng và lợi ích nhận được từ việc cập nhật đồng bộ so với cập nhật không đồng bộ,
2. Đề xuất chiến lược đặt vị trí bản sao và tần suất làm tươi phù hợp với đặc tính này.

1. Phân tích chi phí mạng và lợi ích giữa cập nhật đồng bộ vs. không đồng bộ

Tiêu chí	Cập nhật đồng bộ (Synchronous Replication)	Cập nhật không đồng bộ (Asynchronous Replication)
----------	--	---

Chi phí mạng	Rất cao – mỗi thao tác ghi cần lan tỏa ngay đến tất cả bản sao → chi phí truyền thông lớn, đặc biệt nếu geo-distributed	Thấp hơn – ghi trước vào bản chính, sau đó mới cập nhật bản sao → giảm lưu lượng mạng tức thời
Độ trễ ghi	Cao – do phải chờ phản hồi từ nhiều bản sao trước khi xác nhận hoàn tất	Thấp – xác nhận ghi ngay sau khi cập nhật bản chính
Tính nhất quán	Cao – tất cả bản sao luôn giống nhau (nhất quán mạnh - strong consistency)	Thấp hơn – có khả năng bản sao chưa cập nhật kịp (eventual consistency)
Lợi ích	Đảm bảo tính toàn vẹn dữ liệu ở mọi bản sao (phù hợp với các ứng dụng tài chính, giao dịch)	Phù hợp với hệ thống có yêu cầu ghi nhanh , hoặc không cần dữ liệu cập nhật tức thì ở mọi bản sao
Phù hợp với đặc tính $N \ll M$?	Không phù hợp: cập nhật nhiều nhưng đọc ít → chi phí mạng lãng phí	Phù hợp: hạn chế gửi bản ghi tới các bản sao không được đọc thường xuyên

Kết luận: Trong trường hợp này, **cập nhật không đồng bộ** là lựa chọn tốt hơn vì:

- Giảm chi phí truyền thông không cần thiết
- Chấp nhận độ trễ nhất định trong việc đồng bộ hóa bản sao (không ảnh hưởng nhiều nếu đọc ít)

2. Đề xuất chiến lược đặt vị trí bản sao và tần suất làm tươi phù hợp

a. Chiến lược đặt bản sao:

- **Đặt bản sao gần với nơi có yêu cầu đọc (client location)** để giảm độ trễ nếu có truy cập đọc.
- **Không cần đặt nhiều bản sao cho dữ liệu có tần suất đọc thấp**, tránh lãng phí tài nguyên và băng thông.
- Có thể chỉ duy trì **1–2 bản sao phụ trách ghi**, còn lại là backup phục hồi.

Tối ưu phân bố: đặt bản sao chỉ ở những vùng có nhu cầu đọc thực tế. Nếu có dữ liệu nào không được truy cập thường xuyên, chỉ cần giữ bản chính và một bản sao để đảm bảo khôi phục.

b. Chiến lược làm tươi (update/refresh):

- **Không cần đồng bộ tức thời** sau mỗi lần ghi do đọc ít → chọn mô hình **eventual consistency**
- **Làm tươi theo batch (theo lô):**
 - Ghi vào bản chính ngay lập tức
 - Sau mỗi **khoảng thời gian cố định (ví dụ 1 phút / 5 phút)**, cập nhật các bản sao cần thiết
- **Có thể dùng TTL (Time-To-Live) hoặc timestamp-based policy:**
 - Nếu dữ liệu chưa được đọc trong TTL, không cần cập nhật
 - Chỉ làm tươi bản sao khi có dấu hiệu chuẩn bị được truy cập

e. Hãy đánh giá ưu – nhược điểm của mô hình nhân bản đồng bộ (strong consistency) trong một hệ thống phân tán quy mô toàn cầu (ví dụ: hệ thống thanh toán đa quốc gia).

Sau đó, thiết kế một giải pháp nhất quán lỏng (loose consistency) phù hợp, mô tả:

- 1. Cách lựa chọn mức độ “lỏng” (ví dụ causal, eventual),**
- 2. Cơ chế đảm bảo rằng các cập nhật quan trọng (như giao dịch tài chính) vẫn không bị mất,**
- 3. Cách hệ thống sẽ phục hồi nếu một số bản sao quá cũ so với quy định.**

Đánh giá mô hình nhân bản đồng bộ (strong consistency) trong hệ thống phân tán toàn cầu (ví dụ: hệ thống thanh toán đa quốc gia)

Ưu điểm:

- 1. Đảm bảo tính nhất quán tuyệt đối (strong consistency):**
 - Mọi thao tác đọc đều nhận được dữ liệu mới nhất, bất kể truy cập từ bản sao nào.
 - Đặc biệt quan trọng với các hệ thống tài chính, ngân hàng, thanh toán, nơi không được phép có sai lệch dữ liệu.
- 2. Hỗ trợ các giao dịch phân tán an toàn:**
 - Việc cập nhật được xem là nguyên tử, cho phép thực hiện các giao dịch theo chuẩn ACID.

Nhược điểm:

1. Chi phí đồng bộ cao:

- Mỗi cập nhật phải được lan truyền tới *tất cả các bản sao* trước khi được xác nhận → đòi hỏi đồng thuận toàn cục (ví dụ: dùng thuật toán như Paxos, Raft).
- Dễ dẫn tới độ trễ cao trong môi trường toàn cầu, do khác biệt vị trí địa lý và độ trễ mạng.

2. Khó mở rộng quy mô:

- Khi số lượng bản sao tăng hoặc phân bố rộng về mặt địa lý, hiệu năng hệ thống giảm mạnh do chi phí đồng bộ lớn.

3. Độ sẵn sàng thấp khi có lỗi mạng:

- Một số bản sao không phản hồi → hệ thống buộc phải chờ hoặc từ chối giao dịch để đảm bảo toàn vẹn → ảnh hưởng đến khả năng phục vụ.

Giải pháp nhất quán lỏng (loose consistency) phù hợp

1. Cách lựa chọn mức độ “lỏng”

Đề xuất sử dụng **eventual consistency** kết hợp **causal consistency**:

- **Causal consistency** cho phép các cập nhật liên quan đến nhau (theo quan hệ nhân-quả) được thực hiện theo đúng thứ tự → giúp đảm bảo dữ liệu hợp lý hơn so với chỉ dùng eventual.
- **Eventual consistency** cho phép các bản sao không cần đồng bộ ngay lập tức, nhưng *đảm bảo* rằng cuối cùng tất cả sẽ có cùng trạng thái.

Lý do chọn: Đảm bảo được hiệu năng và độ trễ thấp trong môi trường toàn cầu, nhưng vẫn duy trì tính hợp lý giữa các cập nhật.

2. Cơ chế đảm bảo các cập nhật quan trọng (ví dụ: giao dịch tài chính) không bị mất

a. Phân loại dữ liệu theo mức độ quan trọng:

- **Giao dịch tài chính (critical updates):**
 - Luôn sử dụng **cơ chế quorum** (đa số bản sao đồng thuận).
 - Cập nhật được ghi nhận thành công chỉ khi $\geq N/2+1$ bản sao phản hồi.
 - Có thể áp dụng giao thức **write-ahead log (WAL)** và **giao dịch hai pha (2PC)** cho các thao tác nhạy cảm.

- **Dữ liệu thứ cấp (như trạng thái UI, lịch sử tìm kiếm,...):**

- Sử dụng **eventual consistency** để tối ưu hiệu năng.

b. Cơ chế lưu tạm và retry:

- Các bản sao tạm thời mất kết nối sẽ ghi log cập nhật và gửi lại khi kết nối được khôi phục.

c. Xác minh bằng checksum hoặc version vector:

- Giúp phát hiện nếu một bản sao bị thiếu cập nhật, để tự động khôi phục.

3. Cách hệ thống phục hồi nếu một số bản sao quá cũ

a. Sử dụng đồng bộ nền (background sync):

- Các bản sao khi online lại sẽ yêu cầu các cập nhật bị bỏ lỡ dựa vào:
 - **Vector clock / version vector**
 - **Log-based catch-up** từ các bản sao mới hơn

b. Xác định bản sao lỗi thời (stale replica detection):

- Kiểm tra các bản sao có độ trễ lớn hơn giới hạn cho phép.
- Nếu quá ngưỡng, tạm ngừng phục vụ từ bản sao đó, đánh dấu "đang khôi phục".

c. Phục hồi theo hướng dữ liệu (data healing):

- Dựa trên các log cập nhật (WAL) được giữ tại các bản sao khỏe, truyền các thay đổi thiếu cho bản sao cũ → đảm bảo eventual consistency.

d. Cơ chế làm tươi theo thời hạn (TTL - Time To Live):

- Mỗi bản sao có thời gian hết hạn, khi quá hạn mà không được cập nhật → buộc đồng bộ lại toàn bộ dữ liệu.

Câu 2:

a. Hãy liệt kê và định nghĩa ngắn gọn bốn mô hình nhất quán theo thứ tự thao tác được nhắc đến trong phần 8.2.2.

1. Nhất quán nghiêm ngặt (Strict Consistency)

Mọi thao tác đọc phải trả về **giá trị của thao tác ghi gần nhất (theo thời gian tuyệt đối)** trên cùng một mục dữ liệu.

→ **Yêu cầu thời gian toàn cục**, khó thực hiện trong hệ thống phân tán.

2. Nhất quán tuần tự (Sequential Consistency)

Kết quả thực thi tương đương với việc **các thao tác được thực hiện tuần tự**, theo một thứ tự chung mà tất cả tiến trình đều thấy giống nhau, giữ thứ tự nội bộ của từng tiến trình.

→ Không yêu cầu thời gian tuyệt đối, nhưng vẫn cần đồng thuận về thứ tự thao tác.

3. Nhất quán nhân quả (Causal Consistency)

Nếu một thao tác ghi **gây ra (hoặc ảnh hưởng đến)** một thao tác ghi khác, thì **mọi tiến trình phải thấy thao tác đầu tiên trước**.

→ Các thao tác **không liên quan nhân quả** có thể được thấy theo thứ tự khác nhau.

4. Nhất quán hàng đợi (FIFO / PRAM Consistency)

Mọi tiến trình phải **thấy thao tác ghi từ một tiến trình cụ thể theo đúng thứ tự nó đã thực hiện**, nhưng **không cần thống nhất thứ tự thao tác ghi từ các tiến trình khác nhau**.

→ Dễ triển khai hơn, chỉ cần đảm bảo thứ tự theo từng tiến trình riêng lẻ.

b. Giải thích sự khác nhau giữa nhất quán nghiêm ngặt (strict consistency) và nhất quán tuần tự (sequential consistency). Tại sao mô hình tuần tự yếu hơn nhưng lại thực tế hơn trong hệ phân tán?

1. Sự khác nhau giữa nhất quán nghiêm ngặt và nhất quán tuần tự

Đặc điểm	Nhất quán nghiêm ngặt (Strict)	Nhất quán tuần tự (Sequential)
Định nghĩa	Mỗi thao tác đọc phải trả về giá trị của thao tác ghi gần nhất theo thời gian tuyệt đối .	Kết quả giống như các thao tác được thực hiện theo một thứ tự tuần tự nhất định, thống nhất cho mọi tiến trình .
Dựa vào thời gian toàn cục?	Có — yêu cầu thời gian toàn cục tuyệt đối , để xác định "ghi gần nhất".	Không — không phụ thuộc vào thời gian vật lý , chỉ yêu cầu một thứ tự hợp lệ và thống nhất giữa các tiến trình.
Mức độ đồng bộ	Rất cao — thao tác ghi phải được lan tỏa ngay lập tức đến tất cả bản sao.	Thấp hơn — chỉ cần thống nhất thứ tự thao tác giữa các tiến trình , không yêu cầu tức thời.
Tính mạnh - yếu	Mạnh hơn — đảm bảo nhất quán tuyệt đối.	Yếu hơn — cho phép đan xen thao tác nếu vẫn giữ thứ tự logic.
Thực tế trong hệ phân tán	Khó khả thi — vì cần đồng bộ tuyệt đối trong thời gian thực, phải dùng cam kết phân tán rất tốn kém.	Thực tế hơn — vì không đòi hỏi đồng bộ theo thời gian, phù hợp với độ trễ mạng và sự không đồng bộ giữa các tiến trình.

2. Tại sao mô hình tuần tự yếu hơn nhưng thực tế hơn trong hệ phân tán?

- Mô hình tuần tự **yếu hơn** vì:
 - Không đảm bảo thao tác đọc trả về giá trị mới nhất tuyệt đối.
 - Cho phép các thao tác ghi đan xen nhau **nếu vẫn giữ được thứ tự nhất quán giữa các tiến trình**.
- Tuy nhiên, nó **thực tế hơn trong hệ phân tán** vì:
 - **Không yêu cầu đồng bộ thời gian toàn cục**, vốn khó thực hiện do độ trễ mạng và khác biệt đồng hồ hệ thống.
 - **Không bắt buộc thao tác ghi lan tỏa ngay lập tức**, giúp giảm tải mạng và phù hợp với đặc điểm **mạng không đồng bộ**.
 - Dễ hơn để triển khai với hiệu năng tốt trong các hệ thống có nhiều bản sao và tần suất thao tác cao.

c. Một ứng dụng theo dõi điểm số trực tuyến (live scoring) cho một giải đấu thể thao yêu cầu:

1. Người xem tại bất kỳ bản sao nào đều thấy điểm cập nhật gần nhất trong vòng 2 giây.
2. Không cần đảm bảo thứ tự tuyệt đối của các bàn thắng độc lập trên các trận đấu khác nhau.

Hãy chọn mô hình nhất quán liên tục (continuous consistency) hay nhất quán nhân quả (causal consistency), và giải thích cách bạn cấu hình CONIT (đơn vị nhất quán) hoặc cơ chế vector clock để đáp ứng yêu cầu.

Lựa chọn mô hình: Nhất quán liên tục (Continuous Consistency)

Vì sao không chọn *Nhất quán nhân quả*?

- Mô hình causal consistency rất phù hợp khi các thao tác có liên hệ nhân quả rõ ràng (ví dụ: $A \text{ xảy ra} \rightarrow B \text{ phụ thuộc vào } A$).
- Tuy nhiên, các trận đấu độc lập, nên bàn thắng ở trận A không có quan hệ nhân quả với bàn thắng ở trận B.
- Do đó, việc theo dõi và bảo toàn quan hệ nhân quả là không cần thiết, và còn gây thừa tải nếu áp dụng vector clock cho toàn bộ hệ thống.

Vì sao chọn *Nhất quán liên tục*?

- Mô hình này cho phép nói lỏng tính nhất quán, nhưng vẫn kiểm soát được độ lệch theo thời gian, giá trị và thứ tự.
- Đáp ứng yêu cầu thực tế: “người xem thấy điểm số trong vòng 2 giây” → cấu hình giới hạn độ lệch thời gian.
- Không bắt buộc duy trì thứ tự giữa các thao tác cập nhật độc lập → phù hợp với các trận đấu khác nhau.

Cách cấu hình CONIT để đáp ứng yêu cầu

Sử dụng mô hình CONIT (Consistency Unit) để đo và kiểm soát mức độ nhất quán:

1. Phạm vi đơn vị nhất quán (unit of consistency):

- Cấu hình mỗi trận đấu là một đơn vị CONIT riêng (vì các trận là độc lập).
- Tránh dùng toàn bộ hệ thống làm một CONIT vì sẽ gây trễ không cần thiết.

2. Cấu hình theo 3 trục của CONIT:

Trục đo	Cấu hình phù hợp	Giải thích
Độ lệch thời gian cập nhật (staleness delay)	≤ 2 giây	Mọi bản sao phải thấy điểm số được cập nhật không trễ quá 2 giây sau khi ghi.
Độ lệch giá trị số (numerical divergence)	= 0 (hoặc 1 cập nhật chưa lan truyền)	Vì điểm số là dữ liệu chính xác, sai số phải bằng 0 (mọi người thấy điểm đúng), hoặc cho phép 1 thao tác cập nhật chưa được áp dụng nếu thời gian vẫn < 2 giây.
Khác biệt về thứ tự thao tác (ordering divergence)	Không kiểm soát (nói lỏng)	Vì không yêu cầu thứ tự giữa các trận, có thể cho phép bản sao A thấy “trận 1 ghi bàn trước”, bản sao B thấy “trận 2 ghi bàn trước”.

d. So sánh về hiệu năng và tính dễ triển khai giữa hai mô hình:

- **Nhất quán tuần tự (sequential consistency)**
- **Nhất quán hàng đợi (FIFO consistency)**

Hãy phân tích khi nào bạn nên chọn FIFO thay vì tuần tự, dựa trên đặc tính lưu lượng đọc/ghi và mức độ tương tranh giữa các tiến trình.

1. So sánh mô hình nhất quán tuần tự và FIFO

Tiêu chí	Nhất quán tuần tự (Sequential)	Nhất quán hàng đợi (FIFO)
Định nghĩa	Mọi tiến trình thấy cùng một thứ tự toàn cục của tất cả thao tác đọc/ghi (giữ thứ tự nội bộ tiến trình).	Mỗi tiến trình thấy các thao tác ghi của một tiến trình khác theo đúng thứ tự mà tiến trình đó đã phát sinh, nhưng thứ tự giữa các tiến trình có thể khác nhau.
Yêu cầu về đồng bộ hóa	Cao: cần đồng thuận về thứ tự thao tác toàn cục (phải truyền nhiều thông điệp).	Thấp: chỉ cần duy trì thứ tự cục bộ từ mỗi tiến trình ghi riêng biệt.
Hiệu năng	Thấp hơn do cần đồng bộ nhiều hơn, đặc biệt trong môi trường phân tán.	Cao hơn vì cho phép cập nhật song song giữa các tiến trình.
Tính dễ triển khai	Khó triển khai hơn, cần đảm bảo tất cả tiến trình nhìn thấy cùng một thứ tự thao tác.	Dễ triển khai hơn, có thể dùng Lamport clock để gán nhãn theo tiến trình gửi.
Thích hợp cho	Hệ thống cần độ chính xác cao về thứ tự thao tác giữa các tiến trình.	Hệ thống có tính phân tán mạnh, cần tối ưu hiệu năng, thứ tự thao tác toàn cục không bắt buộc.

2. Khi nào nên chọn FIFO thay vì tuần tự?

Chọn FIFO khi:

- Lưu lượng ghi cao từ nhiều tiến trình
→ FIFO cho phép thao tác ghi diễn ra song song, vì không cần toàn bộ tiến trình đồng thuận về thứ tự toàn cục → giảm độ trễ và tăng hiệu năng.
- Mức độ tương tranh cao giữa các tiến trình ghi
→ Với mô hình tuần tự, cần điều phối để quyết định thứ tự toàn cục → rất tốn tài nguyên. FIFO chỉ cần giữ thứ tự ghi trong từng tiến trình, nên nhẹ hơn.
- Thứ tự giữa các thao tác ghi của các tiến trình khác nhau không quan trọng
→ Ví dụ: Hệ thống cập nhật trạng thái cảm biến, hoặc hệ thống game online nơi mỗi người chơi điều khiển thực thể riêng.

- Ưu tiên hiệu năng hơn nhất quán tuyệt đối về thứ tự
→ Nếu người dùng không cần thấy mọi thao tác ghi theo cùng một trình tự, mà chỉ cần thấy đúng thứ tự từ mỗi nguồn riêng biệt.

Không nên chọn FIFO nếu:

- Ứng dụng đòi hỏi đồng thuận mạnh về thứ tự sự kiện giữa các tiến trình (ví dụ: hệ thống ngân hàng, giao dịch chứng khoán, điều khiển phân tán).
- Cần kiểm tra logic liên tiến trình, ví dụ: tiến trình A ghi “bắt đầu” và tiến trình B ghi “kết thúc”, thì mọi tiến trình phải thấy đúng thứ tự này → FIFO không đủ.

e. Thiết kế một chiến lược nhất quán dữ liệu cho nền tảng chat nhóm đa khu vực (multi-region group chat), với các yêu cầu:

1. Tin nhắn trong cùng một phòng chat phải xuất hiện theo thứ tự gửi (nhất quán tuần tự trên nhóm).
2. Các phòng chat khác nhau có thể xử lý không đồng bộ.
3. Đảm bảo không quá 100 ms độ trễ trung bình khi hiển thị tin nhắn mới.

Trình bày:

- Mô hình nhất quán bạn chọn (ví dụ nhóm con của sequential hoặc causal),
- Cơ chế tổ chức “đơn vị nhất quán” (CONIT) hoặc cơ chế nhãn thời gian,
- Quy trình lan tỏa cập nhật và đồng bộ giữa các bản sao,
- Cách xử lý khi có bản sao trễ quá ngưỡng thời gian cho phép.

1. Mô hình nhất quán chọn dùng:

- Nhất quán tuần tự (Sequential Consistency) áp dụng theo từng phòng chat. Đây là một nhóm con của nhất quán thao tác, đảm bảo mọi thành viên trong cùng một phòng chat đều nhìn thấy tin nhắn theo cùng một thứ tự.
- Áp dụng Nhất quán liên tục (Continuous Consistency) khi giữa các bản sao khu vực chưa đồng bộ hoàn toàn, nhưng độ lệch nằm trong ngưỡng cho phép.

2. Cơ chế tổ chức đơn vị nhất quán (CONIT) và nhãn thời gian:

- Mỗi phòng chat là một đơn vị nhất quán riêng biệt.
- Mỗi tin nhắn được gán một Vector Clock dạng **<t, i>**:

- **t**: thời gian logic (hoặc Lamport timestamp) của tin nhắn.
- **i**: ID node gửi tin nhắn.

Đảm bảo rằng mọi bản sao có thể so sánh thứ tự tin nhắn để tái lập thứ tự hợp lệ khi đồng bộ.

3. Quy trình lan tỏa cập nhật và đồng bộ giữa các bản sao:

Lan tỏa nội vùng (intra-region):

- Mỗi vùng địa lý có một Local Chat Server, dùng bộ nhớ đệm (cache) để hiển thị nhanh tin nhắn mới với độ trễ thấp.
- Khi một người dùng gửi tin:
 - Tin nhắn sẽ được phát tán tới tất cả người dùng trong vùng ngay lập tức ($\leq 10\text{ms}$).
 - Đồng thời đẩy vào hàng đợi gửi đến các vùng khác.

Lan tỏa liên vùng (cross-region):

- Áp dụng giao thức đồng bộ mềm (soft commit) theo định kỳ (20–50ms/lượt).
- Sử dụng kênh riêng cho từng phòng chat để truyền tin nhắn.
- Mỗi bản sao giữ Vector Clock để biết độ trễ và phát hiện thứ tự lệch.

4. Xử lý khi bản sao trễ quá ngưỡng (vi phạm ràng buộc 100ms):

- Đặt ngưỡng CONIT cho độ lệch thứ tự là $u \leq 3$ thao tác và độ lệch thời gian $\leq 100\text{ms}$.
- Nếu độ lệch vượt ngưỡng:
 - Tạm ẩn các tin nhắn mới để tránh hiển thị sai thứ tự.
 - Ưu tiên đồng bộ cưỡng bức bằng cơ chế “push urgent update”.
 - Nếu cần, sử dụng cơ chế hồi phục thứ tự (reordering) theo vector clock trước khi hiển thị.

Câu 3:

a. Định nghĩa khái niệm “nhất quán sau cùng” (eventual consistency):

Nhất quán sau cùng là một mô hình nhất quán dữ liệu trong hệ thống phân tán, trong đó các bản sao dữ liệu có thể không đồng nhất ngay tại thời điểm thực hiện các thao tác, nhưng đảm bảo sẽ trở nên nhất quán sau một khoảng thời gian nếu không có thêm cập nhật mới nào.

Nói cách khác, hệ thống cho phép một mức độ không nhất quán tạm thời giữa các bản sao, nhưng cam kết rằng *cuối cùng*, mọi bản sao sẽ đồng bộ và chứa cùng một dữ liệu.

Khác biệt cơ bản giữa "nhất quán sau cùng" và "nhất quán liên tục":

Tiêu chí	Nhất quán sau cùng	Nhất quán liên tục
Mức độ nhất quán	Không đảm bảo ngay lập tức, chỉ đảm bảo sau một thời gian	Cho phép sai lệch có kiểm soát trong các giới hạn định trước
Khả năng kiểm soát sai lệch	Không kiểm soát độ lệch cụ thể giữa các bản sao	Có thể đo lường và giới hạn độ lệch về giá trị, thời gian hoặc thứ tự cập nhật
Thời điểm các bản sao nhất quán	Không xác định trước, phụ thuộc vào quá trình lan truyền	Luôn duy trì độ lệch trong ngưỡng cho phép, bản sao "gần như" nhất quán
Ứng dụng phù hợp	Dữ liệu ít xung đột, chủ yếu đọc (read-heavy systems)	Ứng dụng yêu cầu phản hồi nhanh nhưng vẫn cần kiểm soát mức độ không nhất quán
Ví dụ minh họa	Mạng xã hội, hệ thống DNS, email	Giá cổ phiếu (sai số không quá 0.01), dữ liệu thời tiết cập nhật mỗi giờ

b. Giải thích mô hình nhất quán đọc đều (read-your-writes consistency):

1. Nguyên tắc hoạt động của nó là gì?
2. Tại sao nó đảm bảo rằng “khách không bao giờ thấy dữ liệu cũ hơn lần đọc trước”?

1. Nguyên tắc hoạt động của mô hình nhất quán đọc đều (Read-Your-Writes Consistency):

Mô hình nhất quán đọc đều yêu cầu rằng nếu một tiến trình đã đọc một phiên bản dữ liệu nào đó, thì các lần đọc tiếp theo của chính tiến trình đó sẽ không bao giờ nhận được dữ liệu cũ hơn. Nghĩa là:

- Khi máy khách (client) thực hiện một thao tác đọc, hệ thống đảm bảo rằng mọi thao tác đọc kế tiếp từ máy khách đó sẽ trả về dữ liệu “ít nhất là mới bằng dữ

liệu đã đọc trước đó”.

- Nếu máy khách đọc từ nhiều bản sao khác nhau (ví dụ đọc trước từ bản sao L1, sau đọc từ bản sao L2), thì bản sao L2 **phải có dữ liệu mới ít nhất bằng bản sao L1 đã đọc trước đó**.

⇒ Điều này đòi hỏi hệ thống cần theo dõi lịch sử đọc của từng client và **đảm bảo không đưa client “quay về quá khứ” trong quá trình đọc dữ liệu**.

2. Tại sao mô hình này đảm bảo rằng “khách không bao giờ thấy dữ liệu cũ hơn lần đọc trước”?

Mô hình này đảm bảo được điều đó nhờ vào các cơ chế sau:

- Khi một client đã từng đọc dữ liệu tại một bản sao (ví dụ L1), thì hệ thống ghi nhận mức độ cập nhật của dữ liệu đó (ví dụ đã đến phiên bản **x2**).
- Khi client đọc từ một bản sao khác (ví dụ L2), hệ thống sẽ **đảm bảo L2 đã được cập nhật ít nhất đến mức **x2** trở lên**, nghĩa là không thể đọc được phiên bản **x1** cũ hơn.

Ví dụ minh họa từ Hình 8.17(a):

- Giả sử thao tác ghi **WS(x1, x2)** được thực hiện lần lượt lên bản sao L1 rồi L2.
- Khi client đọc L1 và nhận được **x2**, sau đó đọc L2 cũng nhận được **x2**, vì L2 đã được cập nhật đầy đủ.
Điều này **thoả mãn mô hình nhất quán đọc đều**.

Trường hợp vi phạm (Hình 8.17(b)):

- Bản sao L2 thực hiện ghi **WS(x2)** nhưng **chưa có ghi **x1** trước đó**, nên cấu trúc dữ liệu có thể bị thiếu liên kết logic.
- Dù L2 có **x2** tại thời điểm đó, nhưng vì thiếu **x1**, nên trong tương lai **L2 vẫn có thể quay về trạng thái cũ hơn như **x1** → vi phạm nhất quán đọc đều**.
Như vậy, client có thể **đọc được dữ liệu “cũ hơn” lần đọc trước đó**, không đảm bảo nguyên tắc.

c. Một ứng dụng di động cho phép người dùng ghi chú (notes) offline và sau đó đồng bộ khi có mạng. Đề xuất cách kết hợp nhất quán ghi đều (monotonic writes) và nhất quán đọc kết quả ghi (read-your-write) để đảm bảo:

- Mọi ghi chú tạo ra ở client luôn được áp dụng theo đúng thứ tự.
- Người dùng luôn thấy ghi chú vừa lưu ngay cả khi chuyển sang một node khác.

1. Đảm bảo thứ tự các ghi chú bằng mô hình nhất quán ghi đều (Monotonic Writes):

Nguyên tắc: Nhất quán ghi đều đảm bảo **mọi thao tác ghi (write)** của một tiến trình (ví dụ client) lên cùng một mục dữ liệu **phải được thực hiện theo đúng thứ tự gốc** mà client đó đã tạo ra.

Áp dụng cho ứng dụng ghi chú:

- Khi người dùng tạo nhiều ghi chú offline (ví dụ Note 1, Note 2, Note 3), hệ thống cần **gửi và ghi các ghi chú lên server theo đúng thứ tự này** khi đồng bộ lại.
- Server (hoặc các bản sao dữ liệu) cần **đảm bảo thứ tự áp dụng ghi chú** đúng như client gửi đi, tránh trường hợp Note 3 được ghi trước Note 1.

Kỹ thuật hỗ trợ:

- Gán **timestamp** hoặc **sequence number** tăng dần cho mỗi ghi chú tại client.
- Server kiểm tra thứ tự trước khi chấp nhận ghi, hoặc **trì hoãn ghi nếu một ghi chú đến mà phụ thuộc ghi chú trước chưa đến**.

2. Đảm bảo người dùng luôn thấy ghi chú vừa lưu bằng mô hình nhất quán đọc kết quả ghi (Read-Your-Writes):

Nguyên tắc: Mô hình này đảm bảo **nếu client đã ghi dữ liệu nào đó thì các lần đọc tiếp theo của client đó sẽ nhìn thấy đúng dữ liệu mình vừa ghi**, kể cả khi đọc từ bản sao khác.

Áp dụng cho ứng dụng ghi chú:

- Sau khi ghi một ghi chú (ví dụ Note 2), nếu người dùng chuyển sang thiết bị khác hoặc node khác, và thực hiện đọc lại, hệ thống **phải đảm bảo dữ liệu hiển thị bao gồm Note 2**.
- Nếu bản sao dữ liệu của node mới chưa được đồng bộ đủ, hệ thống phải **chờ đến khi bản sao đó cập nhật xong dữ liệu ghi gần nhất của client mới** cho phép đọc.

Kỹ thuật hỗ trợ:

- Theo dõi **phiên làm việc (session context)** hoặc **ID người dùng + sequence number** đã ghi gần nhất.
- Khi đọc từ một node mới, yêu cầu node đó đảm bảo đã **đồng bộ đầy đủ dữ liệu đến mức ghi gần nhất của client** trước khi trả về kết quả.

d. So sánh ưu–nhược điểm giữa hai mô hình:

- **Nhất quán đọc đều (Read-Your-Writes)**
- **Nhất quán ghi sau khi đọc (Write-After-Read)**

Trong trường hợp hệ thống đa khu vực với lưu lượng đọc nhiều hơn ghi, hãy phân tích khi nào bạn nên chọn mỗi mô hình, dựa trên:

1. Chi phí đồng bộ và độ trễ,
2. Tính dễ triển khai và phức tạp trên client.

1. Ưu – Nhược điểm

Tiêu chí	Nhất quán đọc đều	Nhất quán ghi sau khi đọc
Ưu điểm	Người dùng không bao giờ thấy dữ liệu lỗi thời – cải thiện trải nghiệm người dùng.	Đảm bảo không ghi đè dữ liệu đã bị cập nhật – tránh mất mát dữ liệu mới nhất.
	Phù hợp cho client hay chuyển vùng, ví dụ đọc ở khu vực A rồi chuyển sang B.	Thích hợp cho các luồng làm việc dựa trên dữ liệu đọc (ví dụ: bình luận sau khi đọc bài).
Nhược điểm	Cần theo dõi phiên bản dữ liệu đã đọc → tăng độ phức tạp đồng bộ trên client hoặc middleware.	Ghi bị trì hoãn nếu dữ liệu chưa đồng bộ hoàn toàn với bản sao mới → tăng độ trễ ghi.
	Có thể yêu cầu thiết lập bộ nhớ tạm hoặc ghi log để duy trì phiên bản đọc trước.	Cần thêm bước kiểm tra phiên bản đọc → làm chậm hiệu năng ghi.

2. So sánh trong hệ thống đa khu vực, đọc nhiều hơn ghi

Yếu tố phân tích	So sánh cụ thể
------------------	----------------

Chi phí đồng bộ	- Read-Your-Writes cần đảm bảo các bản sao sau luôn $\geq$ phiên bản đã đọc $\rightarrow$ có thể cần đợi hoặc định tuyến đến node phù hợp.
	$\rightarrow$ Chi phí cao hơn nếu client chuyển vùng.
	- Write-After-Read chỉ yêu cầu ghi không xảy ra trên bản sao cũ hơn phiên bản đã đọc $\rightarrow$ đồng bộ ít nghiêm ngặt hơn.
Độ trễ	- Read-Your-Writes có thể gây trễ ở bước đọc, nhất là khi chuyển node.
	- Write-After-Read có thể gây trễ ở bước ghi, do phải xác minh hoặc chờ đồng bộ bản sao đến phiên bản đã đọc.
Tính dễ triển khai	- Read-Your-Writes dễ triển khai hơn trong hệ thống có cache client hoặc đọc tại một node cố định.
	- Write-After-Read cần kiểm soát rõ phiên bản trước khi ghi $\rightarrow$ khó hơn trên client không trạng thái.
Phù hợp khi nào?	- Read-Your-Writes phù hợp khi:
	Trải nghiệm người dùng yêu cầu thấy dữ liệu cập nhật ngay.
	Client thường xuyên đọc lại sau khi ghi.
	Người dùng chuyển vùng hoặc dùng nhiều thiết bị.
	- Write-After-Read phù hợp khi:
	Ghi chỉ được phép thực hiện khi đã có nhận thức rõ về trạng thái dữ liệu (ví dụ: sửa comment, phản hồi chuỗi thảo luận).
	Ưu tiên duy trì tính chính xác dữ liệu ghi hơn là tốc độ.

e. Thiết kế một giải pháp nhất quán lấy máy khách làm trung tâm cho nền tảng chat nhóm offline-first:

1. Chọn và kết hợp các mô hình client-centric phù hợp (ví dụ: eventual, read-your-writes, monotonic writes, write-after-read).

2. **Mô tả cơ chế:** khi nào client đọc, khi nào client ghi, và cách lan tỏa cập nhật lên server cũng như các node khác.
3. **Đề xuất cách xử lý xung đột** khi hai client offline cùng ghi đè lên cùng một tin nhắn.
4. **Trình bày cách đánh giá hiệu năng và trải nghiệm người dùng** (độ trễ tối đa, đảm bảo không mất dữ liệu).

Để đảm bảo trải nghiệm nhất quán và mượt mà khi người dùng hoạt động offline, ta cần kết hợp 4 mô hình nhất quán lấy máy khách làm trung tâm như sau:

Mô hình	Mục tiêu sử dụng trong ứng dụng chat
Nhất quán sau cùng (Eventual Consistency)	Đảm bảo dữ liệu giữa các node sẽ giống nhau sau khi đồng bộ hoàn tất.
Nhất quán ghi đều (Monotonic Writes)	Đảm bảo các tin nhắn do một client gửi sẽ được ghi nhận theo đúng thứ tự.
Nhất quán đọc kết quả ghi (Read-Your-Writes)	Người dùng luôn thấy ngay tin nhắn mình vừa gửi, kể cả khi chuyển mạng/node.
Nhất quán ghi sau khi đọc (Write-After-Read)	Ngăn người dùng gửi phản hồi dựa trên tin nhắn cũ chưa được đồng bộ.

2. Cơ chế hoạt động

Ghi:

- Tin nhắn được lưu **tạm thời** vào local storage với timestamp (theo đồng hồ logic hoặc vector clock).
- Mỗi thao tác ghi đều được xếp hàng để đảm bảo **monotonic writes**.
- Khi có kết nối mạng, client sẽ:
 - Gửi toàn bộ các ghi chú/tin nhắn chưa đồng bộ đến server.

- Gắn thêm thông tin: ID người gửi, thời gian gửi, và thông tin phiên bản của tin nhắn cuối cùng đã đọc (để hỗ trợ write-after-read).

Đọc:

- Khi client đọc dữ liệu (lịch sử chat hoặc tin nhắn mới), sẽ:
 - Ưu tiên hiển thị tin nhắn đã ghi trước đó (read-your-writes).
 - Đồng bộ hóa với bản sao gần nhất, đảm bảo **monotonic read**: tin nhắn đọc được trước đó sẽ không biến mất khi chuyển node.

Lan tỏa cập nhật đến server và các node khác:

- Server đóng vai trò trung gian đồng bộ giữa các client và lưu trữ trung tâm.
- Khi client online:
 - Gửi tin nhắn chưa đồng bộ lên server.
 - Nhận các cập nhật mới từ server (dựa trên vector clock hoặc timestamp).
 - Server phát tán đến các client khác qua push hoặc polling.

3. Xử lý xung đột khi nhiều client ghi offline vào cùng một tin nhắn

Trường hợp xung đột: Hai client sửa cùng một tin nhắn trong khi offline (ví dụ: chỉnh sửa nội dung tin nhắn cũ).

Chiến lược xử lý:

- **Sử dụng Vector Clock hoặc CRDTs (Conflict-free Replicated Data Types)** để phát hiện và xử lý xung đột.
- Khi phát hiện xung đột:
 - Nếu chỉnh sửa khác nhau: hiển thị cảnh báo “Xung đột chỉnh sửa” kèm 2 phiên bản và cho phép người dùng chọn giữ bản nào.
 - Nếu có thể hợp nhất (ví dụ: cùng sửa chính tả), sẽ tự động merge.
- Lưu lịch sử phiên bản để rollback hoặc kiểm tra về sau.

4. Đánh giá hiệu năng và trải nghiệm người dùng

Hiệu năng:

Tiêu chí	Phân tích
Độ trễ tối đa	Phụ thuộc vào thời điểm client kết nối lại. Cần tối ưu thời gian sync (ưu tiên tin nhắn mới).
Chi phí đồng bộ	Tương đối thấp nhờ chỉ truyền tin nhắn thay đổi, kết hợp batch gửi khi reconnect.
Khả năng mở rộng	Cao nếu dùng cơ chế vector clock và CRDT để giảm phụ thuộc đồng bộ toàn cục.

Trải nghiệm người dùng:

Vấn đề	Giải pháp
Tin nhắn biến mất khi chuyển vùng mạng	Nhờ Read-Your-Writes và Monotonic Reads, tin nhắn đã đọc luôn hiển thị.
Chậm thấy tin nhắn từ người khác	Dùng push notification hoặc WebSocket để cập nhật theo thời gian thực khi online.
Không mất dữ liệu khi offline	Tin nhắn được lưu cục bộ và retry đồng bộ khi mạng phục hồi.